

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

数论进阶：逆元与组合数

Advanced Number Theory: Modular Inverse and Combinatorics

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 14 日

目录

1. 扩展欧几里得算法
2. 乘法逆元
3. 中国剩余定理
4. 组合数公式法
5. 卢卡斯定理
6. 总结



目录

1 扩展欧几里得算法

2 乘法逆元

3 中国剩余定理

4 组合数公式法

5 卢卡斯定理

6 总结

扩展欧几里得：贝祖定理

- 贝祖定理：对任意整数 a, b，存在整数 x, y 使得 $ax + by = \gcd(a, b)$
- 扩展欧几里得在求 $\gcd$ 的同时解出一组 (x, y)
- 递归边界 $b = 0$ 时，取 $x = 1$，$y = 0$，此时 $ax = a = \gcd$
- 回溯时利用 x, y 互换关系修正解

exgcd 是求逆元、解同余方程、求线性不定方程的基础工具。

代码：exgcd 递归实现


```cpp
// 求 ax + by = gcd(a, b) 的一组整数解，返回 gcd(a, b)
long long exgcd(long long a, long long b, long long& x, long long& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    long long d = exgcd(b, a % b, y, x);   // 交换传入 *
    y -= a / b * x;                        // 回溯修正 *
    return d;
}
```

回溯时先交换 x, y 传入，返回后只需修正 y。

代码：exgcd 解一般不定方程


```cpp
// 求 ax + by = c 的一组解
// 若 c 不能被 gcd(a, b) 整除则无解
bool solveEquation(long long a, long long b, long long c,
                   long long& x, long long& y) {
    long long d = exgcd(a, b, x, y);
    if (c % d != 0) return false;
    long long k = c / d;
    x *= k;
    y *= k;
    return true;
}
```

通解形式：$x + k \cdot b/d$ 与 $y - k \cdot a/d$，k 取任意整数。

目录

1 扩展欧几里得算法

2 乘法逆元

3 中国剩余定理

4 组合数公式法

5 卢卡斯定理

6 总结

三种求逆元的方法对比

方法	复杂度	前提条件	典型场景
扩展欧几里得	$O(\log m)$	$\gcd(a, m) = 1$	模数任意
费马小定理	$O(\log p)$	p 为质数	模数为质数
线性递推	$O(n)$ 预处理	p 为质数	需要 1 到 n 全部逆元

单次用 exgcd 或费马；批量求 1 到 n 的逆元用线性递推。

代码：exgcd 与费马小定理求逆元


```cpp
// 方法一：扩展欧几里得，模数任意
long long invExgcd(long long a, long long m) {
    long long x, y;
    exgcd(a, m, x, y);
    return (x % m + m) % m;                // 调整到非负 *
}

// 方法二：费马小定理，要求 p 为质数
long long invFermat(long long a, long long p) {
    return qpow(a, p - 2, p);
}
```

费马小定理依赖 $a^{p-1} \equiv 1 \pmod{p}$，只在 p 是质数时成立。

代码：线性递推求逆元


```cpp
const int N = 1e6 + 10;
long long inv[N];

// 求 1 到 n 的所有逆元，模数 p 必须是质数
void initInv(int n, long long p) {
    inv[1] = 1;
    for (int i = 2; i <= n; i++)
        inv[i] = (p - p / i) * inv[p % i] % p;   // 线性递推公式 *
}
```

递推式来自 $p \bmod i = p - \lfloor p/i \rfloor \cdot i$，两边同乘 i^{-1} 后移项。

目录

1 扩展欧几里得算法

2 乘法逆元

3 中国剩余定理

4 组合数公式法

5 卢卡斯定理

6 总结

中国剩余定理：问题形式

求解同余方程组：

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \cdots \\ x \equiv a_k \pmod{m_k} \end{cases}$$

前提：$m_1, m_2, \ldots, m_k$ 两两互质。

构造思路：令 $M = \prod m_i$，对每个方程构造一个“只在第 i 个方程里起作用”的项，再相加。

解在模 M 意义下唯一。

代码：中国剩余定理

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
// x = a[i] mod m[i], m[i] 两两互质
long long crt(vector<long long>& a, vector<long long>& m) {
    long long M = 1;
    for (long long mi : m) M *= mi;
    long long ans = 0;
    for (int i = 0; i < (int)a.size(); i++) {
        long long Mi = M / m[i];
        long long inv = invExgcd(Mi % m[i], m[i]);   // Mi 在模 m[i] 下的逆元 *
        ans = (ans + a[i] * Mi % M * inv % M) % M;
    }
    return (ans % M + M) % M;
}
```

每个 M_i 与 m_i 互质，所以逆元一定存在。

目录

1 扩展欧几里得算法

2 乘法逆元

3 中国剩余定理

4 组合数公式法

5 卢卡斯定理

6 总结



组合数公式法：原理

组合数定义式：

$$\binom{n}{m} = \frac{n!}{m!\,(n-m)!}$$

在模 p 意义下，除法要换成乘以逆元：

$$\binom{n}{m} \equiv n! \times (m!)^{-1} \times ((n-m)!)^{-1} \pmod{p}$$

前提：p 为质数，且 n 小于 p。

预处理 $O(n)$，每次查询 $O(1)$；适合 n 不超过 10^6 的场景。

代码：阶乘 + 逆元


```cpp
const int N = 1e6 + 10;
long long fact[N], invFact[N];

// 需要先有 qpow 函数
void initFact(int n, long long MOD) {
    fact[0] = 1;
    for (int i = 1; i <= n; i++)
        fact[i] = fact[i-1] * i % MOD;
    invFact[n] = qpow(fact[n], MOD - 2, MOD);      // 一次快速幂求逆 *
    for (int i = n; i >= 1; i--)
        invFact[i-1] = invFact[i] * i % MOD;       // 倒推阶乘逆元 *
}

long long comb(long long n, long long m, long long MOD) {
    if (m < 0 || m > n) return 0;
    return fact[n] * invFact[m] % MOD * invFact[n-m] % MOD;
}
```

目录

1 扩展欧几里得算法

2 乘法逆元

3 中国剩余定理

4 组合数公式法

5 卢卡斯定理

6 总结



卢卡斯定理：适用场景

当 n, m 很大（比如 10^{18}）但模数 p 较小时，公式法失效，因为 $n!$ 无法直接预处理。

卢卡斯定理给出解法：

$$\binom{n}{m} \equiv \binom{n \bmod p}{m \bmod p} \times \binom{\lfloor n/p \rfloor}{\lfloor m/p \rfloor} \pmod{p}$$

递归展开后，每层的 $n \bmod p$ 与 $m \bmod p$ 都小于 p，可以直接用公式法求。

把大组合数按 p 进制拆位，逐位求解后再相乘。

代码：卢卡斯定理

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
// p 为质数，需先在小范围预处理阶乘与阶乘逆元
// comb 为上面公式法中的 comb，参数均小于 p
long long lucas(long long n, long long m, long long p) {
    if (m == 0) return 1;
    return comb(n % p, m % p, p)
         * lucas(n / p, m / p, p) % p;
}
```

递归深度为 $O(\log_p n)$，每层调用一次 comb；总复杂度 $O(p + \log_p n)$。

目录

1 扩展欧几里得算法

2 乘法逆元

3 中国剩余定理

4 组合数公式法

5 卢卡斯定理

6 总结

总结：复杂度速查

算法	复杂度	前提条件
exgcd	$O(\log n)$	无
费马小定理求逆元	$O(\log p)$	p 为质数
线性递推求逆元	$O(n)$ 预处理	p 为质数
中国剩余定理	$O(k \log M)$	模数两两互质
组合数公式法	$O(n)$ 预处理，$O(1)$ 查询	p 为质数，n 小于 p
卢卡斯定理	$O(p + \log_p n)$	p 为质数

总结：五个高频易错点

- 费马小定理求逆元时，忘记确认模数 p 是质数
- 线性递推求逆元时，初始条件 `inv[1] = 1` 漏写
- CRT 中忘记保证模数两两互质，导致逆元不存在
- 公式法组合数里，n 超过模数 p 后阶乘会变成 0
- 卢卡斯定理里混用大范围和小范围的阶乘数组

逆元是这一章的核心；其他算法几乎都在调用它。

附录：模运算常用结论

- $a^{p-1} \equiv 1 \pmod{p}$（费马小定理，$p$ 为质数）
- $a^{\varphi(m)} \equiv 1 \pmod{m}$（欧拉定理，$\gcd(a, m) = 1$）
- $a^{-1} \equiv a^{p-2} \pmod{p}$（逆元的费马求法）
- $ax \equiv b \pmod{m}$ 有解当且仅当 $\gcd(a, m) \mid b$
- 逆元存在当且仅当 $\gcd(a, m) = 1$

附录：常见提交陷阱

- 逆元函数返回前要（x % m + m）% m，避免出现负数
- 预处理阶乘前先确认数组能装下 n
- CRT 中累加时用 __int128 防止乘法溢出
- 卢卡斯定理中模数 p 必须是小质数，否则预处理会超时
- 多组测试数据时，阶乘数组只需预处理一次